Reviewer Pack — Minimal Hodge Index of Tropical Curves and AC0 Circuit Lower…

Entry c16c1134730f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 21:50:58 UTC

Minimal Hodge Index of Tropical Curves and AC0 Circuit Lower Bounds

Entry ID: `c16c1134730f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 21:50:58 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): AC^0 Circuit Complexity

Statement:

> ['For any Tropical Curve defined by a system of tropical equations over the finite field F_q, the Minimal Hodge Index (H1) is upper bounded by the complexity of an AC0 circuit that computes a related multivariate polynomial.', 'Specifically, for all tropical curves with n ≤ 40 variables, we have: H1(Γ) ≤ c * g(n), where c is a constant and g(n) is the size of the smallest AC0 circuit computing the multivariate polynomial associated with Γ.', 'If there exists a tropical curve with a Minimal Hodge Index that is larger than the complexity of an AC0 circuit computing its associated polynomial, then such an AC0 circuit does not exist.']

Rationale (proposer's reasoning):

> ['The connection between Tropical Geometry and AC0 Circuit Complexity could expose new structural insights into the nature of computation. Tropical Geometry provides a framework for studying problems in algebraic geometry over tropical semirings, while AC0 Circuit Complexity is a well-studied complexity class that can be used to model simple computations. By connecting these fields, we might uncover novel relationships between geometric properties and computational complexity.']

Taxonomy category: `TROPICALGEOMETRY_X_AC0` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b855c5f03b154d9a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For tropical curves with n ≤ 40 variables, if the size of the smallest AC0 circuit computing the associated polynomial is less than or equal to c * g(n), where c is a constant and g(n)

is the Minimal Hodge Index (H1), then support the conjecture. Falsify the conjecture if any curve has an H1 greater than the complexity of its corresponding AC0 circuit.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.90	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Minimal Hodge Index' AND 'Tropical Curves' AND 'AC^0 Circuit Complexity' - 'H1 Tropical Curve' AND 'AC0 Circuit Lower Bounds' AND finite field - tropical geometry AND AC0 complexity AND upper bound Hodge index

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows):
            max_row = i
            for j in range(i+1, rows):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            factor = matrix[i][i]
            for j in range(cols):
                matrix[i][j] /= factor
            for k in range(rows):
                if k != i:
                    factor = matrix[k][i]
                    for j in range(cols):
                        matrix[k][j] -= factor * matrix[i][j]
        return matrix

    def determinant(matrix):
        n = len(matrix)
        det = 0
```

```
        if n == 1:
            return matrix[0][0]
        elif n == 2:
            return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
        else:
            for c in range(n):
                submatrix = [row[:c] + row[c+1:] for row in matrix[1:]]
                det += ((-1) ** c) * matrix[0][c] * determinant(submatrix)
        return det

    def ac0_circuit_size(poly, n):
        # Placeholder function to compute AC0 circuit size
        # This is a dummy implementation and should be replaced with actual logic
        return len(poly)

    def minimal_hodge_index(n):
        # Placeholder function to compute Minimal Hodge Index
        # This is a dummy implementation and should be replaced with actual logic
        return n

    n = random.randint(5, 40)
    poly = sum(random.choice([-1, 1]) * 'x' + str(i) for i in range(n))
    ac0_size = ac0_circuit_size(poly, n)
    hodge_index = minimal_hodge_index(n)

    return {
        "metric_name": "AC0 Circuit Size vs Hodge Index",
        "metric_value": ac0_size,
        "instances_tested": 1,
        "conjecture_holds": ac0_size <= hodge_index,
        "counterexample": "" if ac0_size <= hodge_index else f"Counterexample: AC0 size {ac0_size}
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(r
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.2f} std={std_metric_value:.2f} support_
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"H1 > AC0 circuit size\" first_failing_seed={fir
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0064750a.py", line 81, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0064750a.py", line 63, in run_trial
    poly = sum(random.choice([-1, 1]) * 'x' + str(i) for i in range(n))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating whether the conjecture's support condition was met. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14948
2	preregistration	ollama_remote	glm4:latest	0	0	9809
3	novelty	ollama_remote	glm4:latest	0	0	8734
4	novelty	ollama_remote	glm4:latest	0	0	8760
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13973
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7754
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13333
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10388
9	judge	ollama_remote	glm4:latest	0	0	11734

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 99434 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c16c1134730f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c16c1134730f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c16c1134730f.tar.gz` (if generated)